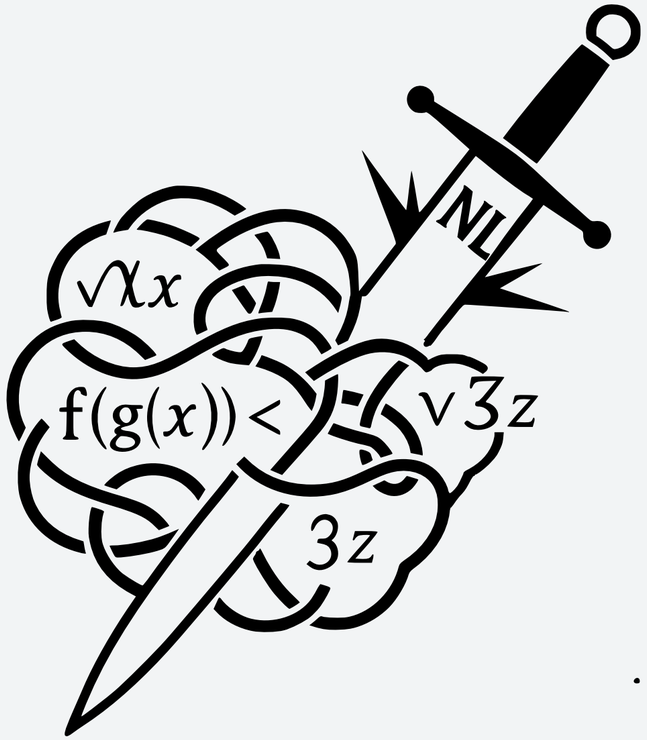
GORDIAN

Defusing Logic Bombs in Symbolic Execution with LLM-generated Ghost Code



DIMITRIOS BOURAS (PKU), **SERGEY MECHTAEV** (PKU)

WHAT ARE LOGIC BOMBS?

THE ANCIENT KNOT BECOMES THE MODERN BOMB

Code regions that are concretely executable, but are extremely hard for symbolic execution to reason through because they generate solver-hostile path conditions or state representations.

SOME EXAMPLES

Library Functions / API Calls

code the solver might not have access to

complex heap/configuration spaces

Program states that depend on many possible object layouts, pointer relations, or environment settings. e.g. Trees, graphs

Logic only described in NL

tax_calc() // this function calculates international tax, encrypts using client's public key

Complex Loops / Complex logic

While-loops with symbolic exit conditions; recursive logic over symbolic structures

Structured inputs

Inputs that must satisfy a specific format like JSON or XML

GORDIAN KNOT

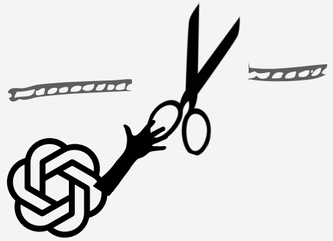
AN INSPIRATION FOR OUT OF THE BOX THINKING IN SYMBOLIC EXECUTION

Problem

Symbolic execution struggles with opaque or complex code

Untangle this complexity

Cut through the complexity.



Current Solutions

- custom logic modeling
- stronger SMT solvers
- elaborate abstraction layers

...

LLMs to the Rescue

Instead of making symbolic reasoning even more complex... Gordian uses LLMs to reinterpret the problem from a new perspective.

GHOST CODE

WHAT IS IT? - HOW DO WE INTERPRET IT?

Traditionally

Auxiliary code added to a program to help reasoning about it

- **invariants**
- **lemmas**
- **specification helpers**

It does not affect program behavior.

Our Interpretation

Ghost code acts as a reasoning aid, not a replacement and is discarded after the analysis

---

- **models solver-hostile fragments**
- **simplifies symbolic reasoning**
- **guides the symbolic executor**
- **modifies Test Cases**

It acts as a bridge between solver reasoning and program semantics

GHOST CODE

HOW DO WE GUARANTEE SOUNDNESS?

The solver struggles
No tests for this path

KLEE

Valid Ghost Code

Test produced for
the interesting path

Invalid Ghost Code

Test produced for
the interesting path
(Not really...)

Ghost code may be approximate and even incorrect since LLMs make mistakes...

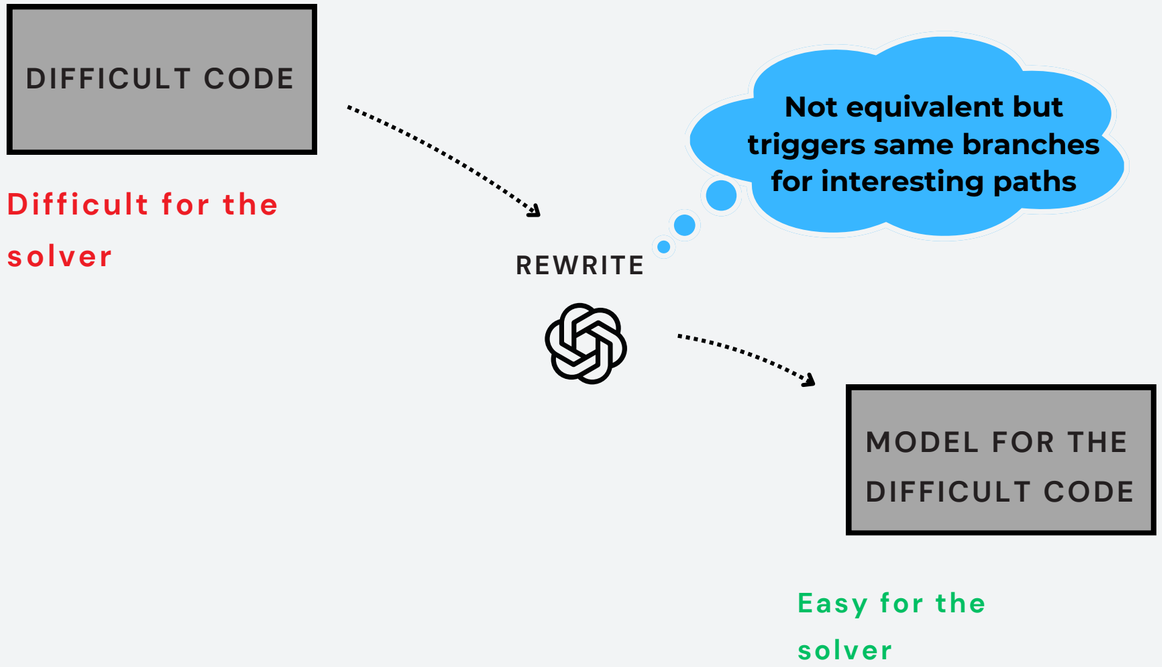
Every discovered test case is validated in the original since the logic bombs are still concretely executable.
0 false positive coverage

INSIDE GORDIAN

PART 1 : GHOST MODELING

IDEA

- Rewrite hostile fragments into surrogates that preserve control-relevant behavior.
- aims for suffix-equivalence; validated by replay



Function operating on floats with 2 decimal places

```c
int check_total(float price, float tax) {
    float total = price * (1.0f + tax);
    if (total > 13.0f)
        return 1;
    return 0;
}
```

Solver-friendly model by scaling to integers

```c
int check_total_model(int price100, int tax100) {
    int total100 = price100 * (100 + tax100);
    if (total100 > 1300 * 100)
        return 1;
    return 0;
}
```

Don’t forget to scale down the test cases!!!

GHOST MODELING

DETAILED EXAMPLE: DEFUSING A REGEX GATE

Guard admits only IDs matching ^[A-Z]{2}[0-9]{4}$ — e.g. LH4711

```c
regex_t re;
regcomp(&re, "^[A-Z]{2}[0-9]{4}$", REG_EXTENDED);
if (regexec(&re, id, 0, NULL, 0) == 0)
    route_flight(id);                    // ← target
```

Model is read off the pattern literal — same branch decision, no automaton

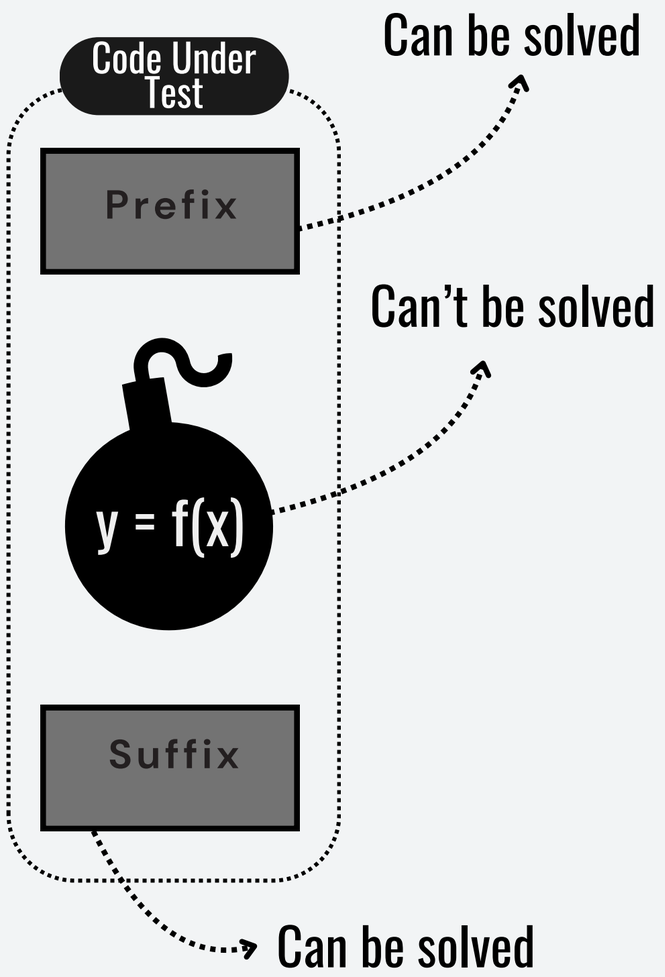
```c
int match_model(const char *s) {
    return s[0]>='A' && s[0]<='Z' && s[1]>='A' && s[1]<='Z'
        && s[2]>='0' && s[2]<='9' && /* ... s[3..5] ... */
        && s[6]=='\0';
}
```

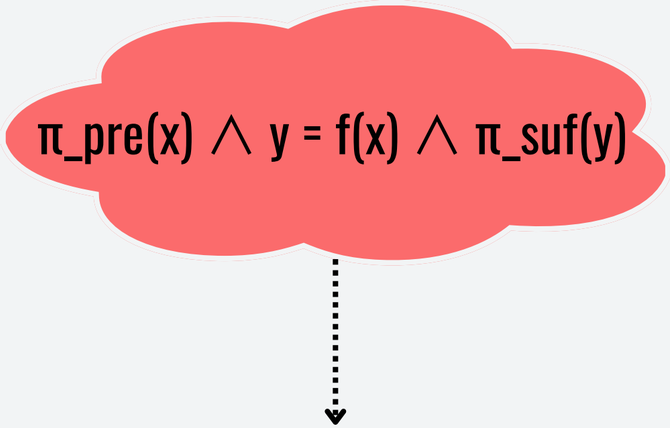
The code is all there — regcomp builds an automaton on the heap at runtime, regexec walks it byte-by-byte over symbolic input. Path explosion inside libc, not one hard constraint.

Regex semantics are LLM world knowledge, not an SMT theory. id stays symbolic — constraints inside route_flight still compose — and every test replays unchanged on the real regexec.

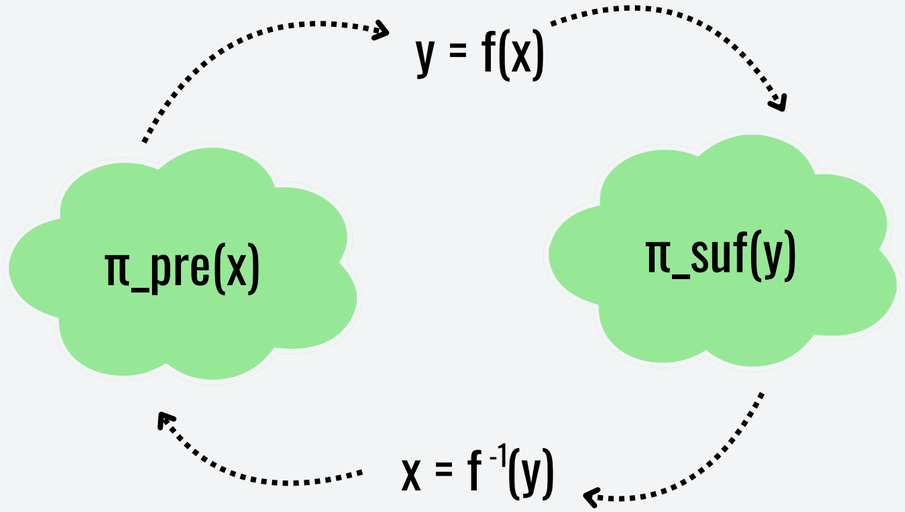
INSIDE GORDIAN

PART 2 : BIDIRECTIONAL CONSTRAINT PROPAGATION





Still Unsolvable



Gordian splits execution around a hard fragment into prefix and suffix constraints, and uses LLM-generated ghost inversion to propagate information between them until both sides become consistent.

INSIDE GORDIAN

BIDIRECTIONAL CONSTRAINT PROPAGATION PIPELINE

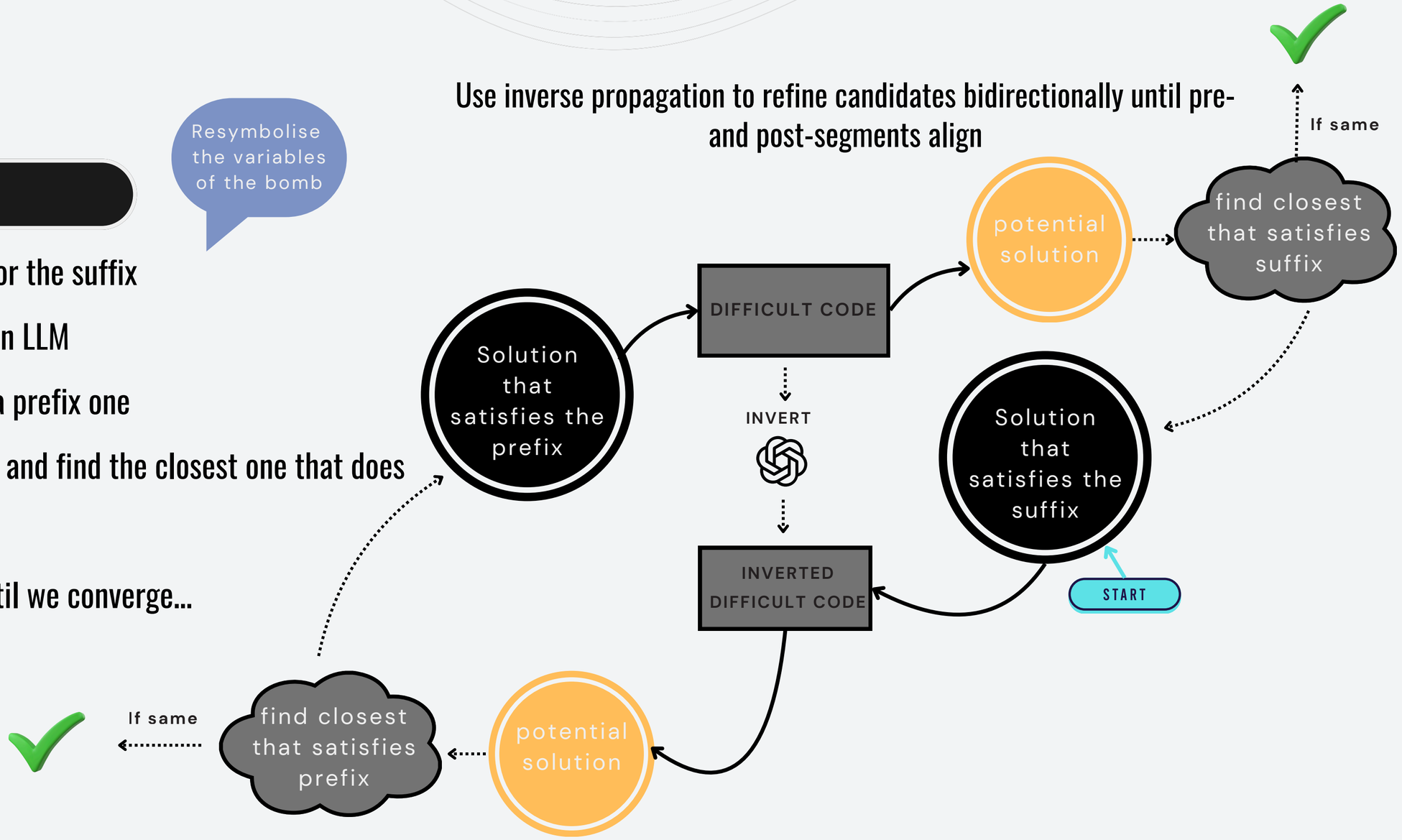
Use inverse propagation to refine candidates bidirectionally until pre- and post-segments align

Resymbolise the variables of the bomb

IDEA

1. Let the solver find a solution for the suffix
2. Invert the difficult code with an LLM
3. Invert the suffix solution into a prefix one
4. If it does not satisfy the prefix and find the closest one that does
5. Propagate it to the suffix

continue until we converge...



INSIDE GORDIAN

EXAMPLE OF BIDIRECTIONAL CONSTRAINT PROPAGATION

Function _ieee754_j0 from fdlibm involving trigonometric operations.

```
hx = __HI(x);
ix = hx & 0x7fffffff;
if (ix >= 0x7ff00000)
  return one / (x * x);
x = fabs(x);
if (ix >= 0x40000000) {
  s = sin(x);
  c = cos(x);
  ss = s - c;
  cc = s + c;
  if (ix < 0x7fe00000) {
    z = -cos(x + x);
    if ((s * c) < 0)
      cc = z / ss;
    else
      ss = z / cc;
    }
}
```

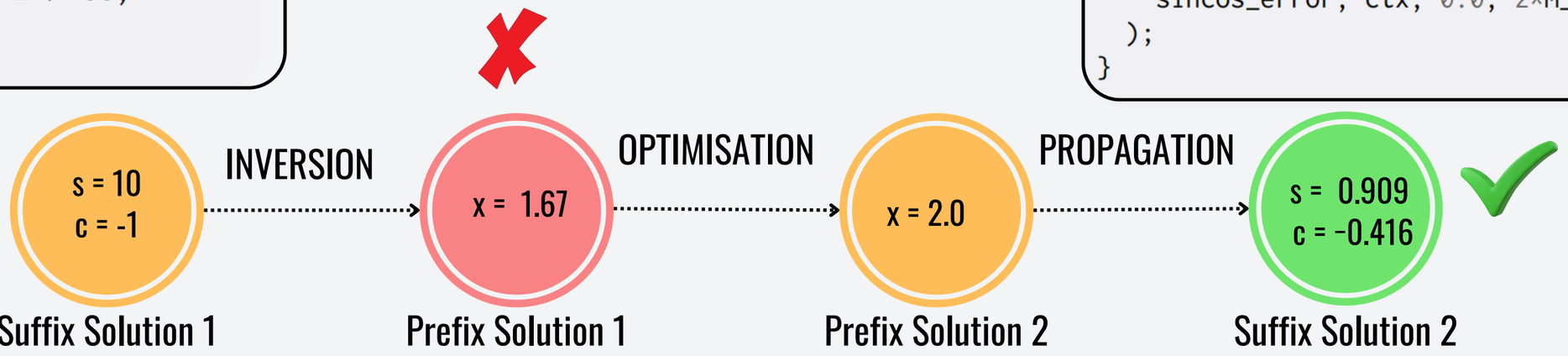
LLM INVERSION

LLM-generated ghost code that inverts s = sin(x); c = cos(x); using golden-section search algorithm

```
double golden_section(double (*f)(double, void *),
                      void *ctx,
                      double a, double b,
                      int max_iter,
                      double tol) {
  algorithm omitted for brevity
}

double sincos_error(double x, void *ctx) {
  double s = sin(x), c = cos(x);
  double ds = s - ((double *)ctx)[0];
  double dc = c - ((double *)ctx)[1];
  return ds * ds + dc * dc;
}

double f_inv(double s, double c) {
  double ctx[2] = {s, c};
  return golden_section(
    sincos_error, ctx, 0.0, 2*M_PI, 200, 1e-12
  );
}
```

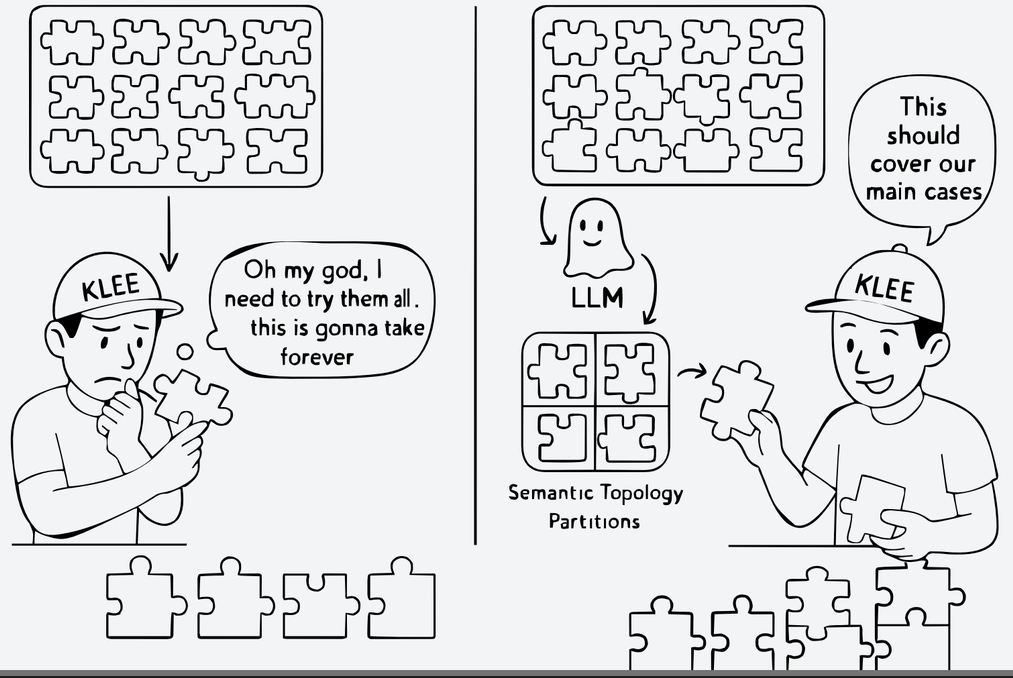


INSIDE GORDIAN

PART 3 : CONSTRAINING HEAP WITH SEMANTIC TOPOLOGIES

IDEA

LLM abstracts heaps into finite semantic shapes (acyclic / cyclic / near-limit).



Emits ghost builders + bounded symbolic choosers to cover each class; internals stay symbolic

Partitions the search space into manageable candidates

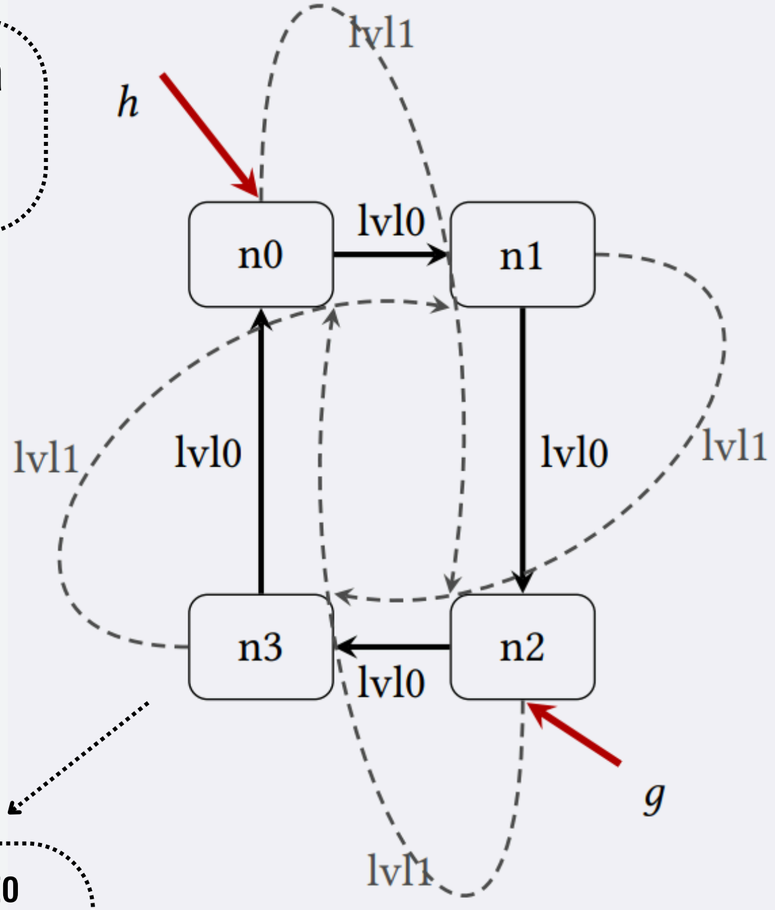
CONSTRAINING HEAP WITH SEMANTIC TOPOLOGIES

EXAMPLE PART 2

```c
typedef struct SNode {
    int val;
    char *name;
    struct SNode *lvl0;
    struct SNode *lvl1;
} SNode;

int proc_if_ring(SNode *h, SNode *g) {
  SNode *a=h ? h->lvl0 : NULL;
  SNode *b=a ? a->lvl0 : NULL;
  SNode *c=b ? b->lvl0 : NULL;
  SNode *d=c ? c->lvl0 : NULL;
  if (!h||!a||!b||!c||!d) return 0;
  if (d!=h) return 0;
  if (h->lvl1!=b||a->lvl1!=c||
      b->lvl1!=d||c->lvl1!=a) return 0;
  SNode *g1 = g ? g->lvl0 : NULL;
  SNode *g2 = g1 ? g1->lvl0 : NULL;
  if (g2!=h) return 0;
  return proc_ring_values(h)
}
```

The function proc_if_ring checks if a skiplist forms a ring, and then processes its values



Ring-shape skiplist required to reach the target function call proc_ring_values

CONSTRAINING HEAP WITH SEMANTIC TOPOLOGIES

EXAMPLE PART 3

```
void constrain_heap(SNode *n0, SNode *n1,
                    SNode *n2, SNode *n3,
                    SNode **h, SNode **g) {
  switch (ξ) {
    case 1: /* broken chain */
      n0->lvl0 = n1; n1->lvl0 = NULL;
      n0->lvl1 = NULL; n1->lvl1 = NULL;
      n2->lvl1 = NULL; n3->lvl1 = NULL;
      *h = n0; *g = n3;
      break;

    three cases omitted for brevity

    case 5: /* ring + g two-steps to h */
      n0->lvl0 = n1; n1->lvl0 = n2;
      n2->lvl0 = n3; n3->lvl0 = n0;
      n0->lvl1 = n2; n1->lvl1 = n3;
      n2->lvl1 = n0; n3->lvl1 = n1;
      *h = n0;
      *g = n2;
      break;
  }
}
```

LLM-generated ghost code that constrains heap with five semantic topologies relevant to the problem context

<u>Why are these not just tests?</u>

- Tests → fully concrete state with fixed all pointers + values
- Our ghost code → partial constraints only heap topology and keeps the remaining data symbolic
- Explore many executions, not one

BASELINES AND BENCHMARKS

5 state of the art tools + 6 diverse benchmarks

ConcoLLMic

Instruments the program to expose execution paths, then given a path uses an LLM to generate inputs and steer concolic exploration toward unexplored branches.

Cottontail

Uses input grammars (e.g. JSON/XML) together with LLM-generated candidates to guide concolic execution for programs with structured inputs.

KLEE - stp

Standard symbolic execution with stp

KLEE-z3

Standard symbolic execution with z3

KLEE- float -z3

Floating-point-aware symbolic execution with Z3.

Logic Bombs
synthetic hard constraints for symex

FDLibM
math library code

jq
JSON query processing

GNU bc
arithmetic expression parser

libexpat
XML parsing

libyaml
YAML parsing

RESULTS

Logic Bombs Dataset

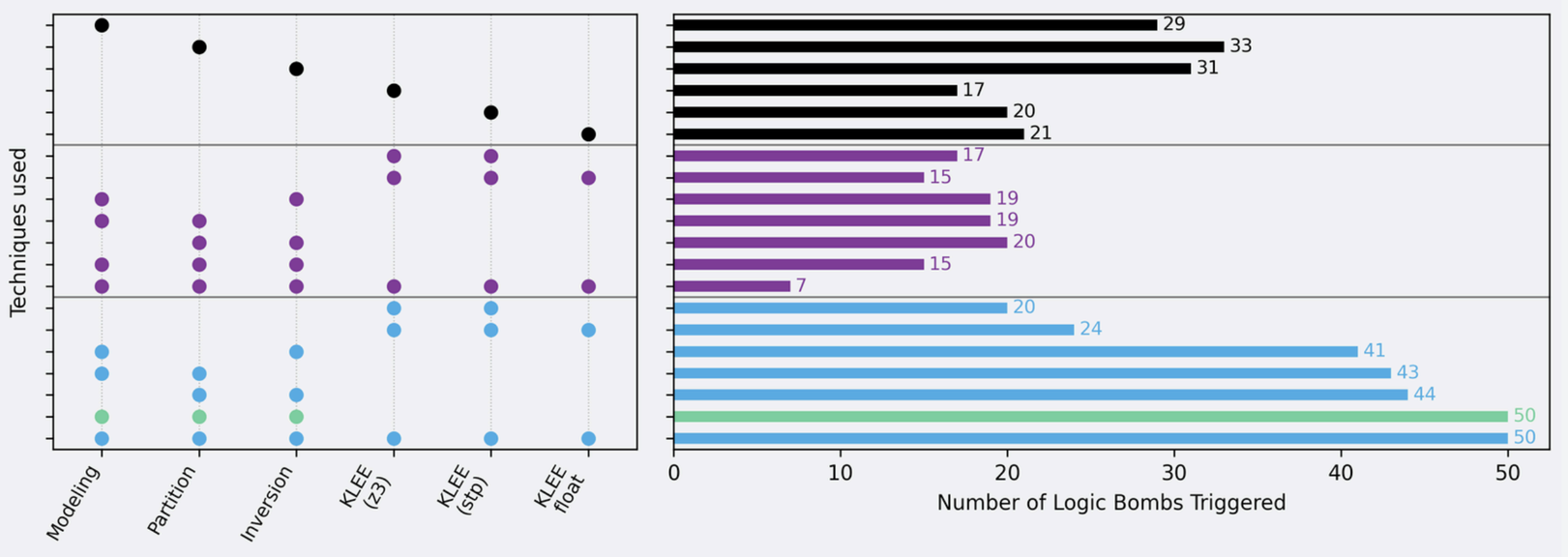
Models

- GPT-5
- Claude-3.7
- DeepSeek-V3.2

LOGIC BOMBS

	Bombs/53	Tokens
KLEE -stp	20	-
KLEE - z3	17	-
KLEE -float -z3	21	-
GORDIAN	**48**	**0.32**
ConcoLLMic	34	3.07

Tokens are in Millions
Results are averages across models



The Gordian techniques are complementary, not redundant

HIGH COVERAGE
At least one technique typically applies to each difficult fragment, enabling symbolic execution to proceed

They provide new capabilities beyond solver improvements, not just incremental gains. Outperform union of KLEE variants.

RESULTS

FDLibM and Structured Inputs

FDLibM

Tool	Coverage %	Tokens(M)
KLEE -stp	70.21	-
KLEE - z3	68.98	-
KLEE -float -z3	57.32	-
GORDIAN	**85.72**	**0.51**
ConcoLLMic	30.25	11.92

Structured-Input Programs (libexpat, jq, GNU bc,libyaml)

Tool	Coverage %	Tokens (M)
KLEE -stp	26.05	-
KLEE - z3	24.97	-
KLEE -float -z3	13.37	-
GORDIAN	**51.99**	**0.42**
ConcoLLMic	7.67	18.07
Cottontail	29.94	4.90

Tokens are in Millions
Results are averages across models

Rerun variability analysis shows that stochastic variation is small relative to Gordian's double-digit coverage gains.

figshare artifacts link

SCAN THE QR FOR ARTIFACTS ACCESS!

THANK YOU FOR YOUR ATTENTION !

KEY TAKEAWAYS

- Do not replace the solver with an LLM! Assist it for solver hostile paths by rerunning the code with LLM generated Ghost code
- Coverage improvement on average by 28.5-115.2% over traditional symbolic execution baselines, and by 74.1–189.8% over LLM-based techniques
- Gordian reduces LLM token usage by 91–96%
- No “false positives”, all test are rerun in the original code!

Pure LLM replacement is not enough; real programs need global, solver-backed reasoning with targeted LLM assistance